

CategorizeIT: Transformer Based Issue Report Classification

Project Team

Adnan Khattak 21I-1141
Saaram Islam 21I-1226
Abdullah Saeed 21I-1230

Session 2021-2025

Supervised by

Mr. Pir Samiullah Shah

Co supervised by

Mr. Irfanullah



Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

September, 2024

Contents

Chapter 1	1
Introduction	2
1.1 Problem Domain	3
1.2 Research Problem Statement	4
Chapter 2	5
Literature Review	5
2.1 Related Research	5
2.1.1 Research Item #1:	5
2.1.1.1 Summary	5
2.1.1.2 Critical Analysis	5
2.1.1.3 Relationship to the Proposed Research	6
2.1.2 Research Item #2:	6
2.1.2.1 Summary	6
2.1.2.2 Critical Analysis	7
2.1.2.3 Relationship to the Proposed Research	7
2.1.3 Research Item #3:	7
2.1.3.1 Summary	7
2.1.3.2 Critical Analysis	8
2.1.3.3 Relationship to the Proposed Research	8
2.1.4 Research Item #4:	8
2.1.4.1 Summary	8
2.1.4.2 Critical Analysis	9
2.1.4.3 Relationship to the Proposed Research	9
2.1.5 Research Item #5:	9
2.1.5.1 Summary	9
2.1.5.2 Critical Analysis	10
2.1.5.3 Relationship to the Proposed Research	10
2.2 Summary of Research Items	10
Chapter 3	11
Proposed Approach	11
3.1 Framework Overview	11
3.2 Methodology	11
3.2.1 Data Preprocessing	11
3.2.2 Model Training	12
3.2.3 Classification and Prediction	12
3.2.3 Evaluation Metrics	12
3.2.4 Architecture Diagram	13
3.3 Conclusion	13
3.4 References	14

Chapter 1

Introduction

Issue reports are a fundamental part of keeping a software system up to date and healthy in the modern age of software development. Issue reports are generally user-generated experiences and include bugs, features or questions with the products functionality. To be resolved in an efficient way, reports must first pass through the intake process so that this data can be processed, categorized as a specific type of request or concern, and sent to the appropriate teams to resolve. With the increasing volume of these reports, manual classification is simply not able to keep up and in turn develop cycles slow or critical fixes are delayed.

To overcome this, CategorizeIT is developed to automatically classify issues based on revolutionary machine learning approaches powered by transforming models. CategorizeIT aims to classify issue reports correctly into predefined categories, Bug, Enhancement, Question and help the management of issues. This automation can assist developers in prioritizing tasks, ensures quicker response times, and enhances overall software quality by reducing the manual effort involved in sorting and assigning issues.

This is done by using natural language processing (NLP) and machine learning algorithms to process the text in issue reports and divert them to the right category. In this way it allows dev teams to worry more about solving the real problems and spending less time managing with sorting huge issue reports.

Like prior research, CategorizeIT also builds on top of pre-trained BERT models for attaining accurate categorization of issues. Issuable automation workflows and productivity enhancements are the cornerstone of this cutting-edge system, a necessity for any software development team in today's era. The project offers a web interface as well to the work for integrating with existing issue tracking systems that make more convenient to use and usable for developers and quality assurance teams.

1.1 Problem Domain

Efficient issue tracking & categorization is important to system maintenance, to a better user experience or to prompt updates when it comes to software development. As the size and complexity of software systems grow, the number of reported issues goes up rapidly as well. These reports take many different forms: from bug descriptions to new features or questions of the user. It is not only time-consuming but also error-prone; that means a lot misclassification of reports, delayed resolutions and you can imagine how useful feature would miss out being built as development resources has been wasted.

The CategorizeIT project tackles this by working to automatically classify issue reports. The issue is in the manual process of sorting and categorizing these reports, taking up time and other

resources that could be better spent improving on actual software. Additionally, incorrect categorization of issues can lead to delays in resolving critical bugs, while less important tasks may be prioritized due to misclassification.

Besides, with the growing importance of agile methodologies in software development, categorization is not something that can wait either. Operations teams need to be able to quickly respond when something needs fixing and not being able to categorize reports correctly, slows down this process. This inefficiency eventually results in dissatisfied customers, security vulnerabilities and decreased software quality among others.

CategorizeIT operates in issue report management from large software development environments trend. CategoryIT aims at using machine learning methods, more specifically transformer-based models to address issues with manual issue classification. Three types of issues—Bug, Enhancement and Question—are automatically identified in issue content to facilitate focused assignment for resource management, response time minimization and over lifecycle improvement.

Through this project, we aim to bridge the gap between traditional manual categorization methods and modern automation technologies while addressing the critical challenge of handling large volumes of issue reports in a scalable and efficient manner.

1.2 Research Problem Statement

One of the problems typical for modern software development is control over a rapidly growing number of issue reports. These reports include multiple types, including bug, enhancement, and support, in the form of user questions. In the past, the typing and categorization besides allocation of such reports used to be done by personnel and this was time-consuming, tended to be erroneous and also time-consuming. When the number of issue reports being received or collected is high, it becomes a lot difficult to proceed with this manual and systematic workflow, thus becoming a strong constraint in the whole software development process. When reports are wrongly or delayed categorized, important bugs stay unfixed, resources are directed to the wrong course and less important issues take priority over more serious ones.

However, it is worth pointing out that, despite using issue tracking systems, most of these systems are still mostly handled in a manual manner, in that, developers and the quality assurance teams will have to sift through the numerous reports. This process is however time consuming and also involves human error and therefore slows the process of solving software problems. Further, the fully manual approach to report categorization hampers the evaluation of the repetitiveness of issues and the optimizing of resources on tasks with the highest degrees of urgency.

The challenge that Categorize IT aims to solve is thus the absence of an effective, precise categorization solution to assist in the classification of issue reports in extensive software development scenarios. The research aims to fill the gap by developing a machine learning-based solution that automates the classification of issue reports into three main categories: It may be classified as Bug, where an application or feature doesn't work as it was intended; Enhancement,

where a user seeks for an improvement to a feature that is already there and Question, a problem which a user poses concerning the application. As such, CategorizeIT will help to minimize the number of manual efforts exerted by developers and quality assurance teams, and will increase the efficiency of the report categorizing process while making sure that important issues are dealt with in the shortest possible amount of time.

Chapter 2

Literature Review

2.1 Related Research

The concerns analysed in the review relate to prior ten years of research around issue report classification in software development, including different methodologies, models and approaches with their advantages, weaknesses and applicability to CategorizeIT.

2.1.1 Research Item #1:

ClassifAI – Automating Issue Reports Classification Using Pre-Trained BERT Language Models

2.1.1.1 Summary

In ClassifAI, we propose a transformer-based approach utilizing the RoBERTa-Large model for categorizing issue reports to Bug, Enhancement, and Question. For the given model, the training and fine-tuning was performed on a large number of issue reports retrieved from open-source projects, and the final F1-score of the model turned out to be 83.2% of the overall evaluated datasets. The study also shows the efficiency of using deep learning to work with text classification in software engineering

2.1.1.2 Critical Analysis

Strengths of the paper:

- The paper utilizes a state-of-the-art BERT-based model (RoBERTa-Large) that excels in capturing contextual meanings in textual data, leading to high classification accuracy.
- Achieves an F1-score of 83.2%, demonstrating strong performance in categorizing software issues.
- The model is fine-tuned on multiple datasets from open-source projects, ensuring that the findings are generalizable and relevant to real-world applications.

Weaknesses of the paper:

- The model requires a large amount of labeled data for effective training, which may limit its applicability in smaller projects or less-documented environments.
- The study does not explore domain-specific versions of BERT, which could potentially improve accuracy in specific fields or programming languages.
- No discussion is provided regarding the computational cost or speed of the model, which may be an issue for large-scale real-time classification systems.

2.1.1.3 Relationship to the Proposed Research

ClassifiAI has acted as a reference point for the knowledge and approaches that will be obtained by the **CategorizeIT** project. CategorizeIT will also use transformer models, with reference to RoBERTa-Large but the model will be optimally designed for use with lesser data and with better time computational characteristics as well.

2.1.2 Research Item #2:

Automated Classification of Software Issue Reports Using Machine Learning Techniques – An Empirical Study

2.1.2.1 Summary

This empirical analysis compares the results of applying simple statistical models, such as Naïve Bayes, Support Vector Machines, and Random Forest on a set of software issue reports to differentiate between Bug and Non-Bug reports. The study employed a set of 7,401 labeled reports and discovered Random Forest as the most efficient, with an F1-score greater than other models benchmarked.

2.1.2.2 Critical Analysis

Strengths of the paper:

- The study provides a comprehensive comparison of several well-established machine learning models, offering valuable insights into their performance on real-world data.
- Random Forest emerged as the most effective model, which demonstrates the potential of traditional machine learning techniques for binary classification tasks.
- The study utilizes a real-world dataset, ensuring that the findings are applicable to practical software development environments.

Weaknesses of the paper:

- The research is limited to binary classification, focusing only on **Bug** and **Non-Bug**, without considering more granular categories such as feature requests or questions.
- Traditional machine learning models require extensive feature engineering and preprocessing, which increases the complexity of the system.
- The study does not explore modern deep learning techniques, which could improve classification accuracy.

2.1.2.3 Relationship to the Proposed Research

CategorizeIT will use improved transformer models for binary classification but this paper offers a reference study to measure the efficiency of standard ML algorithms. CategorizeIT will extend this feature by providing finer control over classification and was designed to be more scalable than the underlying system used in this study.

2.1.3 Research Item #3:

Ticket Tagger – Machine Learning Driven Issue Classification on GitHub

2.1.3.1 Summary

Ticket Tagger is a classifier uses the FastText model to categorize GitHub issue reports under three broad categories namely Bug, Enhancement and Question. The study shows that FastText can be applied to real-time classification as it achieves accuracy of 82.2% for bug reports, 89.4% for enhancements and 78.1% for questions. The model is characterized by high speed and efficiency but failed in cases when there are ambiguous descriptions of issues in the reports.

2.1.3.2 Critical Analysis

Strengths of the paper:

- The FastText model is lightweight and fast, making it suitable for real-time applications in issue classification.
- The model performs well in categorizing bugs and enhancements, achieving high F1-scores in those categories.
- The study focuses on practical implementation, showing how machine learning can be deployed in real-world systems like GitHub.

Weaknesses of the paper:

- The model's accuracy drops significantly for the question category, indicating difficulties in handling more ambiguous language.
- FastText, while efficient, is not as accurate as transformer-based models in capturing the complexity of natural language.
- The study does not provide a detailed evaluation of the model's performance on larger datasets, which could reveal scalability issues.

2.1.3.3 Relationship to the Proposed Research

For the tasks that Ticket Tagger fails to solve due to lack of efficient language processing, CategorizeIT is designed to deliver a better performance thanks to transformer models like BERT. Thus, although CategorizeIT will be ensuring efficiency Improvement efforts will remain primarily directed at improving accuracy for all issues types.

2.1.4 Research Item #4:

Classifying Issue Reports Based on Feature Descriptions Using Deep Learning

2.1.4.1 Summary

This work centers on sorting issue reports through associating them with appropriate software features as described in manuals. Both the issue reports and the feature descriptions are analyzed with CNNs and RNNs in help of which the most important information can be determined. The work gets high accuracy in issue classification where detailed documentation of a domain exists.

2.1.4.2 Critical Analysis

Strengths of the paper:

- The paper proposes an innovative approach that links issue reports to specific software features, improving classification accuracy when documentation is available.
- The deep learning models (CNNs and RNNs) are effective in capturing relationships between issue reports and feature descriptions.
- The model performs well when applied to projects with detailed user manuals, making it useful for well-documented software systems.

Weaknesses of the paper:

- The reliance on detailed documentation limits the applicability of this approach to projects where such documentation is not available.
- CNNs and RNNs are less efficient compared to more recent transformer-based models like BERT, which handle unstructured text more effectively.
- The study does not explore how the model would perform on a broader set of issue categories beyond feature descriptions.

2.1.4.3 Relationship to the Proposed Research

The study's limitation will be addressed in CategorizeIT by using transformer based models which work well with non-coarsely categorized and non-structured information. The emphasis will be placed on the classification of the issue reports according to some set of categories without much elaborate.

2.1.5 Research Item #5:

Few-Shot Issue Report Classification Using Sentence Transformers

2.1.5.1 Summary

In this paper, we address few-shot learning using Sentence-BERT to identify the type of issue reports with little training data. The study also shows an enhancement in F1-score of 2.44% when compared to baseline methods and highlights the strengths of transformer-based models when data is scarce.

2.1.5.2 Critical Analysis

Strengths of the paper:

- The study addresses the challenge of limited labeled data, showing that few-shot learning can improve classification accuracy with minimal training data.
- Sentence-BERT is particularly effective in capturing context and meaning from issue reports, even with small datasets.
- The research provides valuable insights into how transformer models can be adapted for scenarios where data availability is a constraint.

Weaknesses of the paper:

- The model's performance still drops significantly when dealing with ambiguous or vague issue reports, highlighting the need for further improvements.
- Few-shot learning, while effective, may still require significant computational resources for training and fine-tuning transformer models.

- The study does not explore how the model can be scaled for larger datasets or more complex classification tasks.

2.1.5.3 Relationship to the Proposed Research

Several of the few-shot learning approaches discussed in this study will be integrated into CategorizeIT especially in situations where labelled data is limited. However, CategorizeIT will also aim at enhancing the performance of the model in handling or dealing with inconclusive report issues in order to enhance its ability to perform over a large variety of classes of issues.

2.2 Summary of Research Items

Research Item	Dataset Size	Model Used	Categories Classified	Strengths	Weaknesses
[1]	1500+ reports	RoBERTa-Large	Bug, Enhancement, Question	High accuracy, transformer model	Requires large datasets
[2]	7,401 reports	Random Forest, SVM	Bug, Non-Bug	Comprehensive model comparison	Limited to binary classification
[3]	30,000 reports	FastText	Bug, Enhancement, Question	Fast, lightweight	Lower precision for questions
Feature-based Classification (2022)	Feature-specific reports	CNN, RNN	Bug, Feature Request	Links reports to features	Relies on detailed documentation
[5]	Limited labeled data	Sentence-BERT	Multiple categories	Effective with small datasets	Issues with ambiguous reports

This table summarizes the research items discussed, highlighting their contributions and limitations. These insights provide a foundation for the development of **CategorizeIT**, which will address the gaps in existing literature by employing advanced transformer models for efficient and accurate issue report classification.

Chapter 3

Proposed Approach

This chapter gives the framework to solve the research problem that was postulated in CategorizeIT as a means to solve classification of software issue reports issues by applying intelligent techniques. The proposed solution is based on a transformer-based deep learning framework that will classify issue reports into three primary categories: There are three main classifications for a bug which are Bug, Enhancement, and Question. This chapter presents a briefing on the research methodology with regard to the steps, tools and the models that were used to accomplish the goals of the study.

3.1 Framework Overview

CategorizeIT system will function in the form of an issue report processor and classifier based on a pre-trained transformer model which may include BERT. We start with issue report preprocessing, where the reports are cleaned and tokenized followed by training a transformer model on a labeled data set. The system then categorizes the issue reports according to their contents to the available categories as displayed below.

The key components of the proposed approach are:

- **Data Collection and Preprocessing:** The details of inputs are issues reported in the software repositories which are firstly preprocessed by deleting the noise, word tokenizing and normalizing the data. This helps to make the data to be in a correct format required by the model.
- **Model Selection and Training:** The main of CategorizeIT is based on a transformer model like BERT or similar. The basic model is adapted with an aim to enhance its ability to correctly classify text into categories of Bug, Enhancement, and Questions by training it with a pre-labeled dataset of issue reports.
- **Classification:** When trained, this model can take preprocessed issue reports and categorize it into any of the three solutions. This output in turn is preserved in a structure that could be easily fed to an issue management system.
- **Performance Evaluation:** As for the test and assessment of the model's performance, accuracy, precision, recall, and F1-score are well-known benchmarks. Furthermore, the practical applicability of the model under conditions of real-time classifications will be evaluated.

3.2 Methodology

The proposed methodology consists of the following steps:

3.2.1 Data Preprocessing

- **Tokenization:** The text in issue reports is tokenized into smaller units (tokens), which the model can easily process.

- **Text Cleaning:** Noise such as special characters, stopwords, and unnecessary whitespaces is removed.
- **Lowercasing:** All text is converted to lowercase to standardize the input.
- **Lemmatization:** The words in the text are reduced to their base or root forms to prevent variation in word forms from affecting classification.

3.2.2 Model Training

- **Fine-Tuning BERT:** In this study, the pre-trained BERT is adapted to the scope of problem reports. In this way, the transformer model included contextual connection between words in the reports, and learned how to categorize them into the given classes.
- **Cross-Validation:** To enhance the model's generalizability it will be cross validated thus improving on a situation where the model provides pessimistic estimates of the prediction accuracy by over-fitting to the training observations.

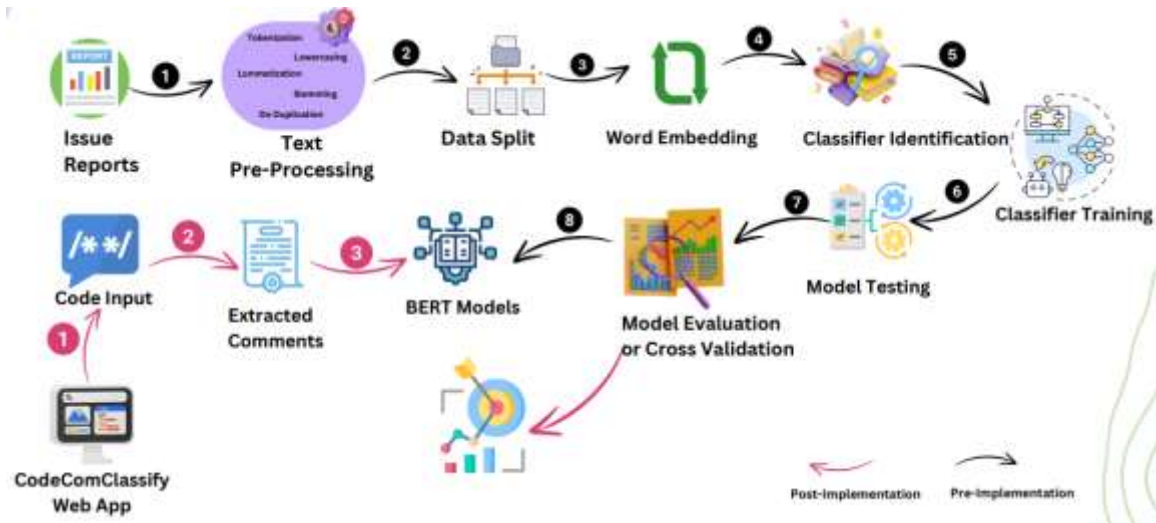
3.2.3 Classification and Prediction

- **Issue Classification:** At the end of the training process the model will be able to predict if the new issue report is a Bug, Enhancement, or Question.
- **Real-Time Processing:** Practicality of the system also means that new issue reports can be quickly consumed and classified in real-time.

3.2.3 Evaluation Metrics

- **Accuracy:** Measures the overall correctness of the classification.
- **Precision:** Evaluates how many of the classified reports were correctly identified.
- **Recall:** Determines how well the model identifies all relevant reports.
- **F1-Score:** Combines precision and recall into a single metric for a balanced evaluation of the model's performance.

3.2.4 Architecture Diagram



3.3 Conclusion

CategorizeIT is being envisioned as an advanced solution, which shall act as an AI-driven solution to facilitate automatic classification of issue reports using machine learning algorithms such as RoBERTa and BERT. Once developed, it will be also convenient to distinguish software issues as bugs, enhancements, and questions with relative high precision which will decrease manual work with these reports.

There will also be flexibility inherent in CategorizeIT to deal with new projects where there could still be minimal labeled data. Using SETFIT and Sentence-BERT, the system will work well with little training due to few-shot learning, making it feasible and versatile for large numbers of software repositories.

As it advances, CategorizeIT could extend and solve other type of issue reports, with many labels or labels not well defined. Thus, in its work, the system will try to achieve high performance, although it will combine distal models such as DistilBERT with classic approaches. In addition, dynamic transformers may be useful for dealing with large report loads in real-time as it may be for projects that have an active set of users and frequent refreshes.

3.4 References

1. **Alam, K.A., Jumani, A., Aamir, H., & Uzair, M. (2024).** ClassifAI: Automating Issue Reports Classification using Pre-Trained BERT Language Models. *Proceedings of ACM/IEEE International Workshop on NL-based Software Engineering (NLBSE '24), Lisbon, Portugal.*
2. **Cho, H., Lee, S., & Kang, S. (2017).** Automated Bug and Enhancement Classification with Naive Bayes. *Information and Software Technology, 142.*
3. **Kallis, R., Di Sorbo, A., Canfora, G., & Panichella, S. (2019).** Ticket Tagger: A Machine Learning Driven Tool for Issue Classification on GitHub. *IEEE International Conference on Software Maintenance and Evolution (ICSME), 406–409.*
4. **Cho, H., Lee, S., & Kang, S. (2022).** Classifying Issue Reports According to Feature Descriptions in a User Manual Based on a Deep Learning Model. *Information and Software Technology, 142.*
5. **Ebrahim, F., & Joy, M. (2024).** Few-Shot Issue Report Classification with Adapters. *Proceedings of ACM/IEEE International Workshop on NL-based Software Engineering (NLBSE '24), Lisbon, Portugal.*